Name: Devarjya
Reg No: 240968040

1) Write a program in CUDA to read MxN matrix. Replace 1st row of this matrix by same elements, 2nd row elements by square of each element and 3rd row elements by cube of each element and so on.

Code:

```
#include <stdio.h>
#include <stdlib.h>

__global__ void powerMatrix(int *a, int m, int n)
{
   int row = blockIdx.x;
   int col = threadIdx.x;

   if (row < m && col < n)
   {
      int idx = row * n + col;
      int val = a[idx];
      int res = 1;

      for (int i = 0; i <= row; i++)
      {
         if (i == 0)
            res = val;      // 1st power
         else
            res *= val;     // subsequent powers
      }

      a[idx] = res;
   }
}

int main(void)
{
   int *a, *t;
   int m, n, i, j;
   int *d_a;
   printf("Name: Devarjya, Reg No: 240968040\n");

   printf("Enter the value of m (rows): ");
   scanf("%d", &m);
   printf("Enter the value of n (cols): ");
   scanf("%d", &n);

   int size = sizeof(int) * m * n;

   a = (int*)malloc(size);
   t = (int*)malloc(size);
```

```
    printf("Enter input matrix:\n");
    for(i = 0; i < m * n; i++)
        scanf("%d", &a[i]);

    cudaMalloc((void**)&d_a, size);

    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);

    powerMatrix<<<m, n>>>(d_a, m, n);

    cudaMemcpy(t, d_a, size, cudaMemcpyDeviceToHost);

    printf("Result matrix is:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
            printf("%d\t", t[i * n + j]);
        printf("\n");
    }

    cudaFree(d_a);
    free(a);
    free(t);

    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 9$ nvcc q1.cu -o q1
pcap1@PC-36:~/Documents/240968040/week 9$ ./q1
Name: Devarjya, Reg No: 240968040
Enter the value of m (rows): 3
Enter the value of n (cols): 3
Enter input matrix:
1
2
3
1
2
3
1
2
3
Result matrix is:
1       2       3
1       4       9
1       8       27
```

2) Write a program in CUDA to add two Matrices using
a. Each row of resultant matrix to be computed by one thread.
b. Each column of resultant matrix to be computed by one thread.
c. Each element of resultant matrix to be computed by one thread.

Code:
a)

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void addMatrixRow(int *a, int *b, int *c, int m, int n)
{
   int row = blockIdx.x;
   int size1 = gridDim.x; // m

   if (row < size1)
   {
      for (int col = 0; col < n; col++)
      {
         int idx = row * n + col;
         c[idx] = a[idx] + b[idx];
      }
   }
}

int main(void)
{
   printf("Name: Devarjya\n");
   printf("Reg No: 240968040\n\n");

   int *a, *b, *c, m, n, i, j;
   int *d_a, *d_b, *d_c;

   printf("Enter the value of m: "); scanf("%d", &m);
   printf("Enter the value of n: "); scanf("%d", &n);

   int size = sizeof(int) * m * n;
   a = (int*)malloc(size);
   b = (int*)malloc(size);
   c = (int*)malloc(size);

   printf("Enter matrix A:\n");
   for(i = 0; i < m * n; i++)
      scanf("%d", &a[i]);

   printf("Enter matrix B:\n");
   for(i = 0; i < m * n; i++)
      scanf("%d", &b[i]);

   cudaMalloc((void**)&d_a, size);
```

```
    cudaMalloc((void**)&d_b, size);
    cudaMalloc((void**)&d_c, size);

    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    addMatrixRow<<<m, 1>>>(d_a, d_b, d_c, m, n);

    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    printf("Resultant Matrix:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
            printf("%d\t", c[i * n + j]);
        printf("\n");
    }

    getchar();
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
    free(a);
    free(b);
    free(c);

    return 0;
}
```

Output

```
pcap1@PC-36:~/Documents/240968040/week 9$ nvcc q2a.cu -o q2a
pcap1@PC-36:~/Documents/240968040/week 9$ ./q2a
Name: Devarjya
Reg No: 240968040

Enter the value of m: 3
Enter the value of n: 3
Enter matrix A:
1
2
3
1
2
3
1
2
3
Enter matrix B:
2
2
2
2
2
2
2
2
2
Resultant Matrix:
3       4       5
3       4       5
3       4       5
```

b)

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void addMatrixCol(int *a, int *b, int *c, int m, int n)
{
   int col = blockIdx.x;
   int size1 = gridDim.x; // n

   if (col < size1)
   {
      for (int row = 0; row < m; row++)
      {
         int idx = row * n + col;
         c[idx] = a[idx] + b[idx];
      }
   }
}

int main(void)
{
   printf("Name: Devarjya\n");
   printf("Reg No: 240968040\n\n");

   int *a, *b, *c, m, n, i, j;
   int *d_a, *d_b, *d_c;

   printf("Enter the value of m: "); scanf("%d", &m);
   printf("Enter the value of n: "); scanf("%d", &n);

   int size = sizeof(int) * m * n;
   a = (int*)malloc(size);
   b = (int*)malloc(size);
   c = (int*)malloc(size);

   printf("Enter matrix A:\n");
   for(i = 0; i < m * n; i++)
      scanf("%d", &a[i]);

   printf("Enter matrix B:\n");
   for(i = 0; i < m * n; i++)
      scanf("%d", &b[i]);

   cudaMalloc((void**)&d_a, size);
   cudaMalloc((void**)&d_b, size);
   cudaMalloc((void**)&d_c, size);

   cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
   cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);
```

```
    addMatrixCol<<<n, 1>>>(d_a, d_b, d_c, m, n);

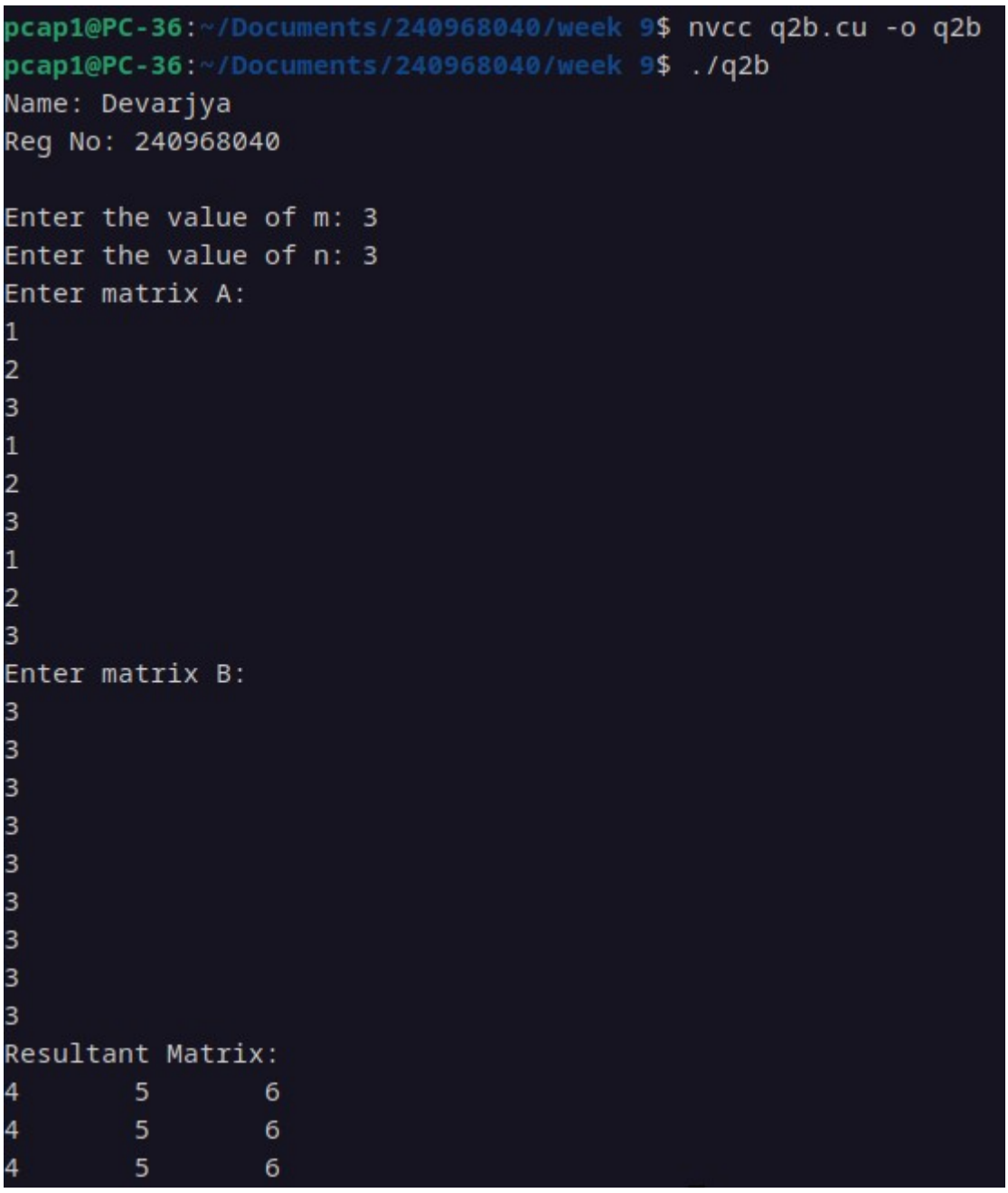
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    printf("Resultant Matrix:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
            printf("%d\t", c[i * n + j]);
        printf("\n");
    }

    getchar();
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
    free(a);
    free(b);
    free(c);

    return 0;
}
```

Output:



c)

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void addMatrixElement(int *a, int *b, int *c, int m, int n)
{
   int row = blockIdx.x;
   int col = threadIdx.x;

   if (row < m && col < n)
   {
      int idx = row * n + col;
      c[idx] = a[idx] + b[idx];
```

```
    }
}

int main(void)
{
    printf("Name: Devarjya\n");
    printf("Reg No: 240968040\n\n");

    int *a, *b, *c, m, n, i, j;
    int *d_a, *d_b, *d_c;

    printf("Enter the value of m: "); scanf("%d", &m);
    printf("Enter the value of n: "); scanf("%d", &n);

    int size = sizeof(int) * m * n;
    a = (int*)malloc(size);
    b = (int*)malloc(size);
    c = (int*)malloc(size);

    printf("Enter matrix A:\n");
    for(i = 0; i < m * n; i++)
        scanf("%d", &a[i]);

    printf("Enter matrix B:\n");
    for(i = 0; i < m * n; i++)
        scanf("%d", &b[i]);

    cudaMalloc((void**)&d_a, size);
    cudaMalloc((void**)&d_b, size);
    cudaMalloc((void**)&d_c, size);

    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    addMatrixElement<<<m, n>>>(d_a, d_b, d_c, m, n);

    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

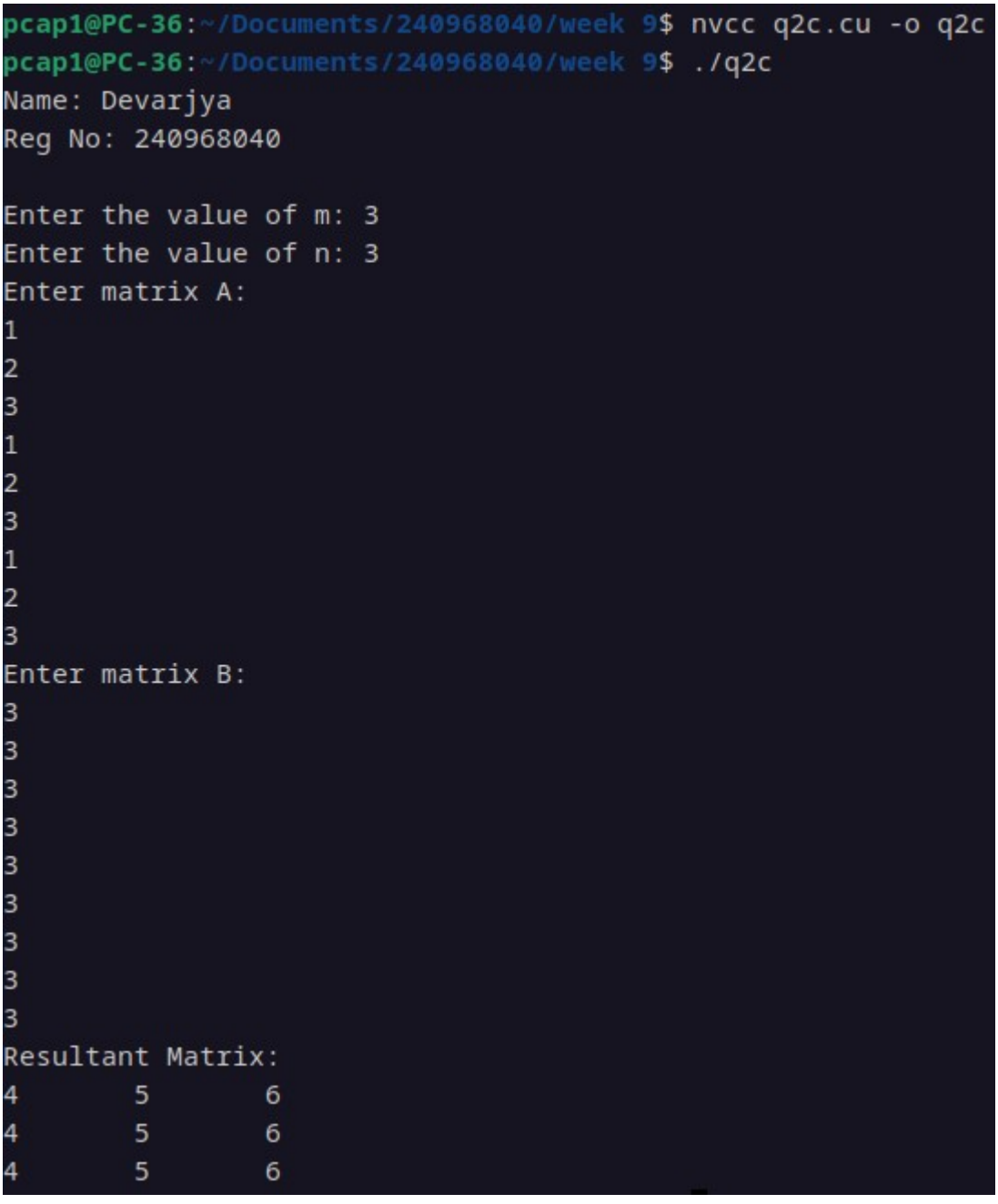
    printf("Resultant Matrix:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
            printf("%d\t", c[i * n + j]);
        printf("\n");
    }

    getchar();
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
    free(a);
    free(b);
```

```
    free(c);

    return 0;
}
```

Output



3) Repeat the above exercise for matrix multiplication.

Code

a)

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void multiplyMatrixRow(int *a, int *b, int *c, int m, int k, int n)
```

```
{
    int row = blockIdx.x; // Each block computes one full row of C

    if (row < m)
    {
        for (int col = 0; col < n; col++)
        {
            int sum = 0;
            for (int i = 0; i < k; i++)
            {
                sum += a[row * k + i] * b[i * n + col];
            }
            c[row * n + col] = sum;
        }
    }
}

int main(void)
{
    printf("Name: Devarjya\n");
    printf("Reg No: 240968040\n\n");

    int *a, *b, *c;
    int m, k, n, i, j;
    int *d_a, *d_b, *d_c;

    printf("Enter rows of Matrix A (m): "); scanf("%d", &m);
    printf("Enter columns of Matrix A / rows of Matrix B (k): "); scanf("%d", &k);
    printf("Enter columns of Matrix B (n): "); scanf("%d", &n);

    int size_a = sizeof(int) * m * k;
    int size_b = sizeof(int) * k * n;
    int size_c = sizeof(int) * m * n;

    a = (int*)malloc(size_a);
    b = (int*)malloc(size_b);
    c = (int*)malloc(size_c);

    printf("Enter Matrix A (%dx%d):\n", m, k);
    for(i = 0; i < m * k; i++)
        scanf("%d", &a[i]);

    printf("Enter Matrix B (%dx%d):\n", k, n);
    for(i = 0; i < k * n; i++)
        scanf("%d", &b[i]);

    cudaMalloc((void**)&d_a, size_a);
    cudaMalloc((void**)&d_b, size_b);
    cudaMalloc((void**)&d_c, size_c);

    cudaMemcpy(d_a, a, size_a, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size_b, cudaMemcpyHostToDevice);
```

```
    // Launch kernel with m blocks and 1 thread per block (one thread per row)
    multiplyMatrixRow<<<m, 1>>>(d_a, d_b, d_c, m, k, n);

    cudaMemcpy(c, d_c, size_c, cudaMemcpyDeviceToHost);

    printf("Resultant Matrix C (%dx%d):\n", m, n);
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
            printf("%d\t", c[i * n + j]);
        printf("\n");
    }

    getchar();
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
    free(a);
    free(b);
    free(c);

    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 9$ nvcc q3a.cu -o q3a
pcap1@PC-36:~/Documents/240968040/week 9$ ./q3a
Name: Devarjya
Reg No: 240968040

Enter rows of Matrix A (m): 3
Enter columns of Matrix A / rows of Matrix B (k): 3
Enter columns of Matrix B (n): 3
Enter Matrix A (3x3):
1
2
3
1
2
3
1
2
3
Enter Matrix B (3x3):
2
2
2
2
2
2
2
2
2
Resultant Matrix C (3x3):
12      12      12
12      12      12
12      12      12
```

b)

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void multiplyMatrixCol(int *a, int *b, int *c, int m, int k, int n)
{
    int col = blockIdx.x; // Each block computes one full column of C

    if (col < n)
    {
        for (int row = 0; row < m; row++)
```

```
        {
            int sum = 0;
            for (int i = 0; i < k; i++)
            {
                sum += a[row * k + i] * b[i * n + col];
            }
            c[row * n + col] = sum;
        }
    }
}

int main(void)
{
    printf("Name: Devarjya\n");
    printf("Reg No: 240968040\n\n");

    int *a, *b, *c;
    int m, k, n, i, j;
    int *d_a, *d_b, *d_c;

    printf("Enter rows of Matrix A (m): "); scanf("%d", &m);
    printf("Enter columns of Matrix A / rows of Matrix B (k): "); scanf("%d", &k);
    printf("Enter columns of Matrix B (n): "); scanf("%d", &n);

    int size_a = sizeof(int) * m * k;
    int size_b = sizeof(int) * k * n;
    int size_c = sizeof(int) * m * n;

    a = (int*)malloc(size_a);
    b = (int*)malloc(size_b);
    c = (int*)malloc(size_c);

    printf("Enter Matrix A (%dx%d):\n", m, k);
    for(i = 0; i < m * k; i++)
        scanf("%d", &a[i]);

    printf("Enter Matrix B (%dx%d):\n", k, n);
    for(i = 0; i < k * n; i++)
        scanf("%d", &b[i]);

    cudaMalloc((void**)&d_a, size_a);
    cudaMalloc((void**)&d_b, size_b);
    cudaMalloc((void**)&d_c, size_c);

    cudaMemcpy(d_a, a, size_a, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size_b, cudaMemcpyHostToDevice);

        multiplyMatrixCol<<<n, 1>>>(d_a, d_b, d_c, m, k, n);

    cudaMemcpy(c, d_c, size_c, cudaMemcpyDeviceToHost);

    printf("Resultant Matrix C (%dx%d):\n", m, n);
```

```
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
            printf("%d\t", c[i * n + j]);
        printf("\n");
    }

    getchar();
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
    free(a);
    free(b);
    free(c);

    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 9$ ./q3b
Name: Devarjya
Reg No: 240968040

Enter rows of Matrix A (m): 3
Enter columns of Matrix A / rows of Matrix B (k): 3
Enter columns of Matrix B (n): 3
Enter Matrix A (3x3):
1
2
3
1

3
2
1
2
3
Enter Matrix B (3x3):
2
2
2
2
2
2
2
2
2
Resultant Matrix C (3x3):
12      12      12
12      12      12
12      12      12
```

c)

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void multiplyMatrixElement(int *a, int *b, int *c, int m, int k, int n)
{
    int row = blockIdx.x;
    int col = threadIdx.x; // Each thread computes a single element of C

    if (row < m && col < n)
    {
        int sum = 0;
        for (int i = 0; i < k; i++)
        {
            sum += a[row * k + i] * b[i * n + col];
        }
        c[row * n + col] = sum;
    }
}

int main(void)
{
    printf("Name: Devarjya\n");
    printf("Reg No: 240968040\n\n");

    int *a, *b, *c;
    int m, k, n, i, j;
    int *d_a, *d_b, *d_c;

    printf("Enter rows of Matrix A (m): "); scanf("%d", &m);
    printf("Enter columns of Matrix A / rows of Matrix B (k): "); scanf("%d", &k);
    printf("Enter columns of Matrix B (n): "); scanf("%d", &n);

    int size_a = sizeof(int) * m * k;
    int size_b = sizeof(int) * k * n;
    int size_c = sizeof(int) * m * n;

    a = (int*)malloc(size_a);
    b = (int*)malloc(size_b);
    c = (int*)malloc(size_c);

    printf("Enter Matrix A (%dx%d):\n", m, k);
    for(i = 0; i < m * k; i++)
        scanf("%d", &a[i]);

    printf("Enter Matrix B (%dx%d):\n", k, n);
    for(i = 0; i < k * n; i++)
        scanf("%d", &b[i]);

    cudaMalloc((void**)&d_a, size_a);
```

```
    cudaMalloc((void**)&d_b, size_b);
    cudaMalloc((void**)&d_c, size_c);

    cudaMemcpy(d_a, a, size_a, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size_b, cudaMemcpyHostToDevice);

        multiplyMatrixElement<<<m, n>>>(d_a, d_b, d_c, m, k, n);

    cudaMemcpy(c, d_c, size_c, cudaMemcpyDeviceToHost);

    printf("Resultant Matrix C (%dx%d):\n", m, n);
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
            printf("%d\t", c[i * n + j]);
        printf("\n");
    }

    getchar();
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
    free(a);
    free(b);
    free(c);

    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 9$ 3^C
pcap1@PC-36:~/Documents/240968040/week 9$ ./q3c
Name: Devarjya
Reg No: 240968040

Enter rows of Matrix A (m): 3
Enter columns of Matrix A / rows of Matrix B (k): 3
Enter columns of Matrix B (n): 3
Enter Matrix A (3x3):
1
2
3
4
5
6
7
8
9
Enter Matrix B (3x3):
3
3
3
3
3
3
3
3
3
Resultant Matrix C (3x3):
18      18      18
45      45      45
72      72      72
```

4) Write an CUDA program that reads a matrix A of size M*N and produces a output matrix B of same size such that it replaces all the non-border elements(numbers in bold) of A with its equivalent 1's complement and remaining elements same as matrix A.

Code:

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void onesComplementMatrix(int *a, int *b, int m, int n)
{
   int row = blockIdx.x;
   int col = threadIdx.x;
```

```
    if (row < m && col < n)
    {
        int idx = row * n + col;

        if (row > 0 && row < m - 1 && col > 0 && col < n - 1)
        {
            b[idx] = ~a[idx];
        }
        else
        {
            b[idx] = a[idx];
        }
    }
}

void printBinaryWithoutLeadingOnes(int n)
{
    int start = -1;
    for (int i = 31; i >= 0; i--)
    {
        if (((n >> i) & 1) == 0)
        {
            start = i;
            break;
        }
    }

    if (start == -1)
    {
        printf("1");
        return;
    }

    for (int i = start; i >= 0; i--)
    {
        printf("%d", (n >> i) & 1);
    }
}

int main(void)
{
    printf("Name: Devarjya\n");
    printf("Reg No: 240968040\n\n");

    int *a, *b;
    int m, n, i, j;
    int *d_a, *d_b;

    printf("Enter the value of m (rows): ");
    scanf("%d", &m);
    printf("Enter the value of n (cols): ");
```

```
    scanf("%d", &n);

    int size = sizeof(int) * m * n;

    a = (int*)malloc(size);
    b = (int*)malloc(size);

    printf("Enter input matrix A:\n");
    for(i = 0; i < m * n; i++)
        scanf("%d", &a[i]);

    cudaMalloc((void**)&d_a, size);
    cudaMalloc((void**)&d_b, size);

    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);

    onesComplementMatrix<<<m, n>>>(d_a, d_b, m, n);

    cudaMemcpy(b, d_b, size, cudaMemcpyDeviceToHost);

    printf("\nResultant Matrix B:\n");
    for(i = 0; i < m; i++)
    {
        for(j = 0; j < n; j++)
        {
            if (i == 0 || i == m - 1 || j == 0 || j == n - 1)
            {
                printf("%d\t", b[i * n + j]);
            }
            else
            {
                printBinaryWithoutLeadingOnes(b[i * n + j]);
                printf("\t");
            }
        }
        printf("\n");
    }

    getchar();
    getchar();

    cudaFree(d_a);
    cudaFree(d_b);
    free(a);
    free(b);

    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 9$ gedit q4.cu
pcap1@PC-36:~/Documents/240968040/week 9$ nvcc q4.cu -o q4
pcap1@PC-36:~/Documents/240968040/week 9$ ./q4
Name: Devarjya
Reg No: 240968040

Enter the value of m (rows): 4
Enter the value of n (cols): 4
Enter input matrix A:
1
2
3
4
6
5
8
3
2
4
10
1
9
1
2
5

Resultant Matrix B:
1       2       3       4
6       010     0111    3
2       011     0101    1
9       1       2       5
^C
```